such as \w for words, \D for non-digits, and character classes like [a-z] for lowercase ASCII Latin and [^c] for any character that isn't lowercase ASCII 'c'. Let's say we wanted to assert that in a word constructed of Latin alphabet characters, 'k' does not follow 'c'. We use a pattern such as /[a-zA-Z]+c[^k]/ but this actually means "One or more Latin alphabet characters, followed by c, followed by any character that isn't k", not "c without a following k" – a subtle but important difference, as the following demonstrates:

```
preg_match_all('/[a-zA-Z]+c[^k]/', 'Disc Dock', $matches);
print_r($matches);
```

```
Output:
    [0] => Array
        (
            [0] => Disc
        )
```

On first glance this looks fine; the string 'Disc' doesn't end in 'ck', and there's no match on 'Dock'. But let's look closer:

```
print bin2hex($matches[0][0]);
```

```
Output:
4469736320
```

We actually have five bytes in the match; the end byte is *0x20*, which is ASCII space: we've captured the space after 'Disc' because of the character class at the end of the pattern. Things are even more problematic if we do this:

```
preg_match_all('/[a-zA-Z]+c[^k]/', 'Dock Disc', $matches);
print_r($matches);
```

```
Output:
    [0] => Array
        (
        )
```

By moving 'Disc' to the end the string, we fail to match anything because there are no characters at all after the final 'c'. The PRCE engine cannot move beyond the 'c' to discover if the next character is "not a k". For our purposes we need to use a (negative)